Contents

Guide 12 — Testing & Validation, the simple version

How Mobile Match Algeria is checked for correctness — in plain words, with the files and ready answers for the jury.

---

1. The idea, in three sentences

Correctness is guarded in two places: **automated unit tests** check the deterministic logic (the recommendation engine, the parsers, the search tokeniser, the formatting helpers), and a **data-validation gate** rejects bad scraped records before they ever reach the database. The first protects the code; the second protects the data.

---

2. The test suite (Vitest)

- **121 unit tests** in total, across four areas: the **recommendation engine**, the **scraping parsers and validation**, the **search-token parser**, and the **formatting utilities**. All pass.
- **Frozen HTML fixtures.** The parser tests run against **real operator-page snapshots** stored in the repository, so the scrapers are tested for regressions **without hitting the live websites** — the tests are fast and stable.
- **Deterministic logic = analytical validation.** Because the recommendation engine always gives the same output for the same input, each scenario test **predicts the result from the rules first**, then asserts the engine matches (budget ceiling, data coverage, the priority weighting, the empty-result edge case).

---

3. The data-validation gate (`validateOffer`)

Tests check the code; this checks the **data**. Every scraped offer must pass a set of sanity rules before it is stored — price and data within plausible ranges, data value not equal to the price (a common parsing mistake), required fields present, valid plan type, and so on. Records that fail are **dropped, not saved**, so a parsing glitch on an operator page cannot poison the catalogue.

---

4. Multi-account functional testing

Beyond unit tests, the app was exercised with several accounts at once to confirm **user isolation**: one account's session, profile, saved plans, and notifications are never visible to another. This is documented in Chapter 4 and complements (does not replace) a formal usability study.

---

5. What is *not* tested (honest)

- No end-to-end / browser UI tests — the validation is at the unit and data level.
- No formal user study with external testers (the FICHE's 50-tester target) — documented as future work; no satisfaction numbers are invented.

Stating this plainly is a strength: it shows you know the boundary of what was verified.

---

6. The files

- **lib/tests/recommendation.test.ts** — the recommendation engine scenarios.
- **lib/tests/search-tokens.test.ts** and **utils.test.ts** — the search parser and the formatting helpers.
- **lib/scraper/tests/validate.test.ts** and **parsers.test.ts** — validation + parsers against frozen fixtures.
- **lib/scraper/validate.ts** — the `validateOffer` gate used at scrape time.

---

7. Jury questions — ready answers

- **How do you know the recommendation engine is correct?** It is deterministic, so each test derives the expected ranking from the rules and asserts the engine produces it — no guessing.
- **How do you test the scrapers without the live sites?** Against frozen HTML snapshots of the real pages stored in the repo, so the parsers are checked for regressions reliably.
- **What stops bad scraped data from being stored?** The `validateOffer` gate: a record that fails the sanity rules is discarded rather than saved.
- **How many tests, and of what?** 121 unit tests covering the recommendation engine, the scraping parsers/validation, the search tokeniser, and the formatting utilities.
- **What are the testing gaps?** No end-to-end UI tests and no external user study — both identified as future work, with nothing fabricated.

This, with guides 09–11, completes the supporting-subsystem set (auth, architecture, security, testing) on top of the eight core-concept guides.